SC1001: Introduction to Computational Thinking

Week 11: Accumulators and Asymptotic Complexity

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

Overview

1. Compare recursive, tail-recursive, and iterative accumulation
2. Generate binary strings recursively
3. Add constraints to recursive generation
4. Practice asymptotic notation

Problem 1: List Length

Reimplement a version of Python's built-in `len` function for lists.

Write three versions:

- Recursive, without an accumulator
- Tail recursive, with an accumulator
- Iterative, with no recursion

List Length: Plain Recursion

```
def list_length1(L):
    if L:
        return 1 + list_length1(L[1:])
    return 0
```

List Length: Plain Recursion

```
def list_length1(L):
    if L:
        return 1 + list_length1(L[1:])
    return 0
```

The recursive call handles the rest of the list; this call adds one.

List Length: Accumulator

```
def list_length2(L):
    return list_length2_helper(L)

def list_length2_helper(L, n=0):
    if L:
        return list_length2_helper(L[1:], n + 1)
    return n
```

List Length: Accumulator

```
def list_length2(L):
    return list_length2_helper(L)

def list_length2_helper(L, n=0):
    if L:
        return list_length2_helper(L[1:], n + 1)
    return n
```

The helper carries the count forward instead of waiting to add after recursion returns.

List Length: Iteration

```
def list_length3(L):
    n = 0
    for item in L:
        n += 1
    return n
```

List Length: Iteration

```
def list_length3(L):
    n = 0
    for item in L:
        n += 1
    return n
```

Iteration makes the accumulator visible as a local variable.

Problem 2: Count Vowels

Write a function that counts the vowels in a string.

Write the same three versions:

- Recursive, without an accumulator
- Tail recursive, with an accumulator
- Iterative, with no recursion

Count Vowels: Plain Recursion

```
def count_vowels1(S):
    if not S:
        return 0
    vowels = "aeiouAEIOU"
    if S[0] not in vowels:
        return count_vowels1(S[1:])
    return 1 + count_vowels1(S[1:])
```

Count Vowels: Plain Recursion

```
def count_vowels1(S):
    if not S:
        return 0
    vowels = "aeiouAEIOU"
    if S[0] not in vowels:
        return count_vowels1(S[1:])
    return 1 + count_vowels1(S[1:])
```

The first character determines whether this call adds one.

Count Vowels: Accumulator

```
def count_vowels2(S):
    return count_vowels2_helper(S)

def count_vowels2_helper(S, n=0):
    if not S:
        return n
    vowels = "aeiouAEIOU"
    if S[0] not in vowels:
        return count_vowels2_helper(S[1:], n)
    return count_vowels2_helper(S[1:], n + 1)
```

Count Vowels: Accumulator

```
def count_vowels2(S):
    return count_vowels2_helper(S)

def count_vowels2_helper(S, n=0):
    if not S:
        return n
    vowels = "aeiouAEIOU"
    if S[0] not in vowels:
        return count_vowels2_helper(S[1:], n)
    return count_vowels2_helper(S[1:], n + 1)
```

The count changes only when the first character is a vowel.

Count Vowels: Iteration

```
def count_vowels3(S):
    vowels = "aeiouAEIOU"
    n = 0
    for char in S:
        if char in vowels:
            n += 1
    return n
```

Warm-Up: All Binary Strings

Before adding constraints, write a recursive function that returns all binary strings of length n.

For $n = 3$, the output should contain:

$$000, 001, 010, 011, 100, 101, 110, 111$$

Use a prefix argument or another way to remember the choices made so far.

All Binary Strings: Base Case

```
def generate_binary_strings(n):
    return generate_binary_strings_helper(n)

def generate_binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]
```

All Binary Strings: Base Case

```
def generate_binary_strings(n):
    return generate_binary_strings_helper(n)

def generate_binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]
```

When no positions remain, the prefix is a complete answer.

All Binary Strings: Branches

```
def generate_binary_strings(n):
    return generate_binary_strings_helper(n)

def generate_binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]

    results = []
    results.extend(generate_binary_strings_helper(n - 1, prefix + "0"))
    results.extend(generate_binary_strings_helper(n - 1, prefix + "1"))
    return results
```

All Binary Strings: Branches

```
def generate_binary_strings(n):
    return generate_binary_strings_helper(n)

def generate_binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]

    results = []
    results.extend(generate_binary_strings_helper(n - 1, prefix + "0"))
    results.extend(generate_binary_strings_helper(n - 1, prefix + "1"))
    return results
```

Each open position branches into a 0 choice and a 1 choice.

Problem 3: Binary Strings Without 00

A binary string uses only `0` and `1`.

Write a recursive Python function that takes an integer $n \geq 1$ and returns all binary strings of length n without consecutive `0`s.

Start from the warm-up solution, then decide when the `0` branch is allowed.

No Consecutive Zeros: New Test

```
def binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]

    results = []
    if prefix == "" or prefix[-1] == "1":
        results.extend(binary_strings_helper(n - 1, prefix + "0"))
```

No Consecutive Zeros: New Test

```
def binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]

    results = []
    if prefix == "" or prefix[-1] == "1":
        results.extend(binary_strings_helper(n - 1, prefix + "0"))
```

The 0 branch is safe at the start or after a 1.

No Consecutive Zeros: Complete Function

```
def binary_strings(n):
    return binary_strings_helper(n)

def binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]

    results = []
    if prefix == "" or prefix[-1] == "1":
        results.extend(binary_strings_helper(n - 1, prefix + "0"))
    results.extend(binary_strings_helper(n - 1, prefix + "1"))
    return results
```

No Consecutive Zeros: Complete Function

```
def binary_strings(n):
    return binary_strings_helper(n)

def binary_strings_helper(n, prefix=""):
    if n == 0:
        return [prefix]

    results = []
    if prefix == "" or prefix[-1] == "1":
        results.extend(binary_strings_helper(n - 1, prefix + "0"))
    results.extend(binary_strings_helper(n - 1, prefix + "1"))
    return results
```

The 1 branch always stays valid.

Problem 4: Hamming Weight

The Hamming weight of a binary string is the number of 1s in it.

Write a recursive function that takes integers $n \geq 1$ and $c \geq 0$ and returns all binary strings of length n with Hamming weight at most c.

Bounded Hamming Weight: State

```
def binary_strings_hw(n, c):
    return binary_strings_hw_helper(n, c)

def binary_strings_hw_helper(n, c, prefix="", hw_so_far=0):
    if n == 0 and hw_so_far <= c:
        return [prefix]
```

Bounded Hamming Weight: State

```
def binary_strings_hw(n, c):
    return binary_strings_hw_helper(n, c)

def binary_strings_hw_helper(n, c, prefix="", hw_so_far=0):
    if n == 0 and hw_so_far <= c:
        return [prefix]
```

The helper remembers both the prefix and the number of 1s used.

Bounded Hamming Weight: Branches

```
def binary_strings_hw_helper(n, c, prefix="", hw_so_far=0):
    if n == 0 and hw_so_far <= c:
        return [prefix]

    results = []
    if hw_so_far < c:
        results.extend(
            binary_strings_hw_helper(n - 1, c, prefix + "1", hw_so_far + 1)
        )
    results.extend(binary_strings_hw_helper(n - 1, c, prefix + "0", hw_so_far))
    return results
```

Bounded Hamming Weight: Branches

```
def binary_strings_hw_helper(n, c, prefix="", hw_so_far=0):
    if n == 0 and hw_so_far <= c:
        return [prefix]

    results = []
    if hw_so_far < c:
        results.extend(
            binary_strings_hw_helper(n - 1, c, prefix + "1", hw_so_far + 1)
        )
    results.extend(binary_strings_hw_helper(n - 1, c, prefix + "0", hw_so_far))
    return results
```

The 1 branch is allowed only while the count remains below the limit.

Problem 5: Big O

Give the big O notation for each worst-case operation count.

1. $n + 12 \log n$
2. 23
3. $n \log n$
4. $\log_5 n + \log_{10} n$
5. $\sqrt{n^{64}} + n^{32} + 994$

Big O Solutions

Expression	Dominant term	Big O
$n + 12 \log n$	n	$O(n)$
23	1	$O(1)$
$n \log n$	$n \log n$	$O(n \log n)$
$\log_5 n + \log_{10} n$	$\log n$	$O(\log n)$
$\sqrt{n^{64}} + n^{32} + 994$	n^{32}	$O(n^{32})$

Big O Solutions

Expression	Dominant term	Big O
$n + 12 \log n$	n	$O(n)$
23	1	$O(1)$
$n \log n$	$n \log n$	$O(n \log n)$
$\log_5 n + \log_{10} n$	$\log n$	$O(\log n)$
$\sqrt{n^{64}} + n^{32} + 994$	n^{32}	$O(n^{32})$

Constants, lower-order terms, and log bases do not change the asymptotic class.

Takeaways

- Accumulators store progress explicitly.
- Helper functions keep accumulator details out of the public interface.
- Recursive generation often builds a prefix and branches.
- Big O keeps the dominant growth rate.